

ChatGPT-CLI — Project Synopsis & Detailed Roadmap

ChatGPT-CLI — Project Synopsis & Detailed Roadmap

Nice — you've chosen a powerful, practical project. Below is a complete, developer-friendly **project proposal + step-by-step roadmap** for building a **Gemini-style ChatGPT CLI** in **pure Java** with **local MongoDB** storage. It covers goals, system architecture, data models, module breakdown, interaction flows, security, testing, deliverables and a phase-by-phase implementation plan you can follow end-to-end.

Project Title

ChatGPT-CLI — Terminal AI Assistant (Java + MongoDB)

1. Project Overview & Objectives

Goal: Build a modern, terminal-based AI assistant that communicates with the OpenAI Chat API, executes shell commands, edits files, and persists conversation history in a local MongoDB database. The tool should feel like a developer assistant (Gemini CLI style): fast, interactive, and extensible.

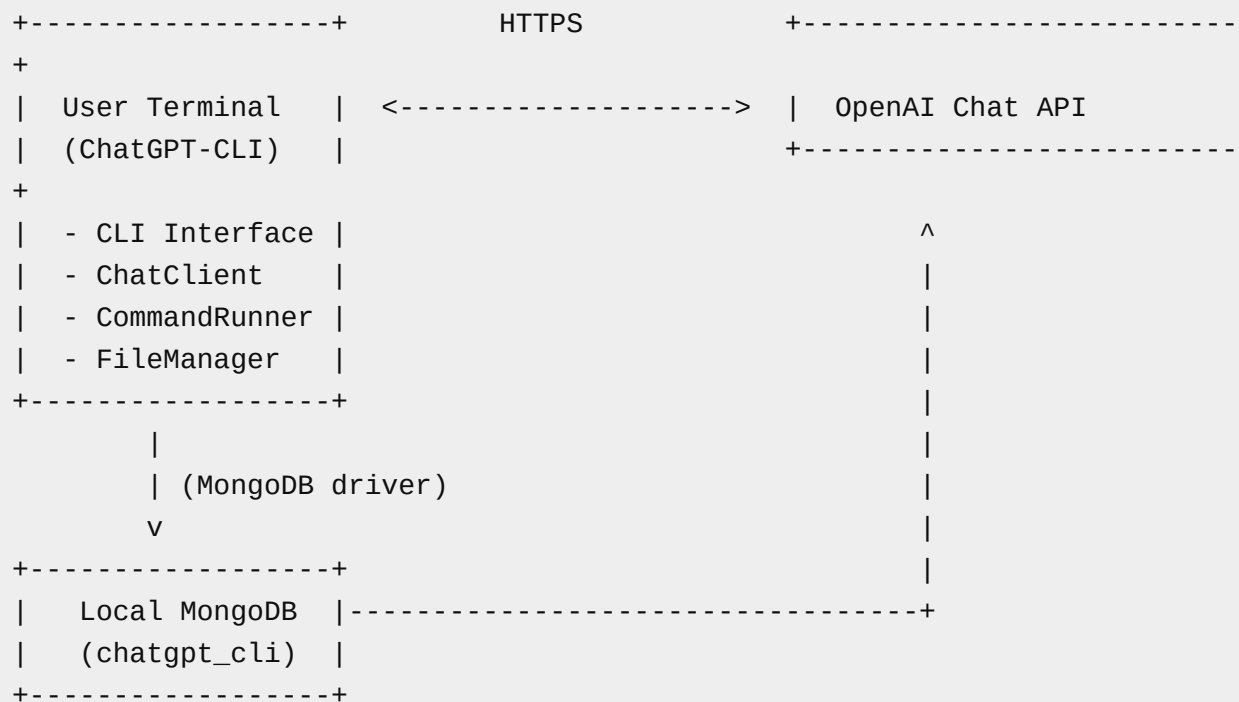
Objectives

- Create a responsive CLI that accepts free text, special command triggers, and structured requests.
- Connect securely to the OpenAI API and display formatted responses.
- Execute shell commands safely and show their output.
- Provide file operations (view, edit, save) driven by AI prompts.
- Persist chat history (sessions & messages) in local MongoDB for recall, search and audit.
- Offer helpful UX (colored output, command suggestions, history browsing).
- Keep the implementation framework-light (pure Java + small libs) so you understand end-to-end.

Non-Goals (for v1)

- No web UI initially.
- Not production-grade security (no production key management, no multiuser auth).
- Not distributed or concurrent multiuser server.

2. High-Level System Architecture



Components

- `CLI` — main program loop & command parsing; responsible for input, local features, showing responses.
- `ChatClient` — handles HTTP calls to OpenAI/Gemini, streaming support, retries.
- `CommandRunner` — executes `!` shell commands with `ProcessBuilder`, returns outputs/errors.
- `FileManager` — read/edit/save files; provides content to `ChatClient` as context for file edits.
- `MongoStorage` — persists sessions & messages; supports query, list, delete history.
- `Config` — loads `.env` (API keys, settings).

- `Utils` — formatting (JColor), suggestion engine (Apache Commons Text), JSON helpers (Gson).
-

3. Tech Stack (final)

- **Java** 21+ (use modern features)
 - **Build:** Maven
 - **HTTP:** `java.net.http.HttpClient` (supports streaming)
 - **JSON:** Gson
 - **Env:** dotenv-java (optional nice to have)
 - **Terminal Colors:** JColor
 - **DB:** MongoDB (local server)
 - **Mongo Driver:** `mongodb-driver-sync`
 - **Command Execution:** `ProcessBuilder`
 - **Suggestion/Spellcheck:** Apache Commons Text (Levenshtein)
 - **Logging:** `java.util.logging` (or SLF4J if you prefer)
-

4. Main Features (v1 → vN)

Core (v1)

- Interactive chat loop (multi-turn).
- Calls to OpenAI Chat Completion endpoint.
- Save each message (user/assistant) to MongoDB in a session document.
- `!` prefix executes shell commands (`!ls` , `!git status`) safely and returns output.
- Save and load chat sessions (`:save` , `:load <sessionId>` , `:history`).
- Basic file operations: `open <file>` , `save <file>` , `edit <file>` (edit via prompts or open local editor).
- Colored output and nice CLI prompts.
- `.env` for API key.

Useful Additions (v2)

- Command auto-suggestions (typo correction).

- Streaming responses (show assistant typing).
- Attach local file contents as context (when user requests edit).
- Tagging and searching sessions in MongoDB.
- Rate-limit handling, retries, backoff.

Advanced (v3+)

- Plugin tools (code runner, patch generator).
- Multi-model support and model selection.
- Auth + user settings, encryption of API key.
- Packaging as native binary (GraalVM).

5. Data Models (MongoDB)

Database: `chatgpt_cli`

Collections:

- `sessions`
- `messages` (optional separate collection if you prefer normalization)

`sessions` document example

```
{
  "_id": "session_2025_10_29_2215",
  "createdAt": "2025-10-29T22:15:00Z",
  "title": "Explain Java Streams",
  "tags": ["java", "streams"],
  "messages": [
    { "role": "user", "content": "Explain Java Streams", "ts": "2025-10-29T22:15:00Z" },
    { "role": "assistant", "content": "Java Streams allow ...", "ts": "2025-10-29T22:15:01Z" }
  ]
}
```

Alternatively store messages as separate documents:

- `messages: { sessionId, role, content, ts, messageId }` — easier to query across sessions.

Indexes

- `sessions.createdAt` (descending)
 - `messages.sessionId` + `messages.ts`
 - Text index on `messages.content` (for searching)
-

6. API Interaction (OpenAI Chat)

- Use `HttpClient` POST to `/v1/chat/completions` (or the current endpoint).
- Request body (example):

```
{
  "model": "gpt-4o-mini",
  "messages": [
    {"role": "system", "content": "..."},
    {"role": "user", "content": "Explain Java Streams"}
  ],
  "max_tokens": 800
}
```

- Parse the choices array, extract assistant content. Use Gson to map responses.
- Optionally support streaming (read response as stream and print progressively).

Important: Respect API quota & add retries/backoff on 429 or temporary 5xx.

7. CLI Design & Commands

Invocation

`$ java -jar chatgpt-cli.jar` → interactive REPL

Or `java -jar chatgpt-cli.jar --prompt "Explain X"` (one-shot)

Special in-CLI commands (suggested)

- `:help` — display commands & usage
- `:exit` — quit & optionally save session
- `:history` — list session ids/titles (from MongoDB)
- `:load <sessionId>` — load previous session into memory
- `!<shell-command>` — run shell command, e.g. `!ls -la`
- `open <file>` — print file contents to console (and offer to attach to prompt)
- `save <file>` — write current assistant answer to a file
- `edit <file>` — ask AI to modify a file (AI returns new content; ask to save)
- `:clear` — clear in-memory messages (not DB unless confirm)
- `:config` — show current model & settings

Interaction examples

- `> write a bash script to delete *.log files` → assistant returns code → `save cleanup.sh` → `!bash cleanup.sh` to run.
-

8. Module / Class Breakdown

- `Main` — bootstrap, reads config, starts CLI.
 - `Config` — loads `.env` (API key, default model, Mongo URI).
 - `CLI` — REPL loop; parse lines and dispatch to `CommandHandler` or `ChatService`.
 - `ChatService` / `ChatClient` — constructs API request, sends to openai, receives response (with streaming option).
 - `CommandHandler` — detect `!` commands and call `CommandRunner`.
 - `CommandRunner` — executes shell commands using `ProcessBuilder`; captures stdout/stderr and exit code; returns formatted result.
 - `FileManager` — read/write files, maintain backups before overwriting.
 - `MongoStorage` — connect to MongoDB, CRUD for sessions/messages.
 - `Formatter` — apply JColor coloring & pretty printing for code blocks (detect ``` blocks, apply different style).
 - `SuggestionEngine` — (optional) checks unknown commands and suggests close matches based on Levenshtein distance.
 - `Logger` — central place for logs and diagnostics.
-

9. Security Considerations

- Store API key in `.env` (not hardcoded). Suggest `.gitignore` exclude `.env`.
 - For local dev, consider encrypting the key (optional) or using OS key store later.
 - Sanitize inputs before running shell commands (avoid naive injection if you provide command templates).
 - When saving files, create backups (`.bak`) before overwriting.
 - Limit command execution scope or prompt user for confirmation for dangerous commands (e.g., `rm -rf /`).
 - No production sensitive data in chat logs; warn the user about storing secrets.
-

10. Testing & QA

- **Unit tests** for: `SuggestionEngine`, JSON serialization, `FileManager` read/write (mock FS where appropriate).
 - **Integration tests** for: `ChatClient` (mock HTTP responses), `MongoStorage` (use a test DB), `CommandRunner` (run safe commands).
 - **Manual tests**: Run end-to-end interactive flows (chat → save → load → run command).
 - Use test sessions that don't call the real API (use mocked responses) for CI.
-

11. Deliverables (Concrete)

- Source code (Maven project) with clear package structure.
 - `README.md` with install & run instructions, screenshots of sample sessions.
 - Example `.env.example` (no secrets).
 - Sample chat sessions preloaded (optional).
 - Unit & integration test suite.
 - Optional: one-file runnable `jar` packaging.
-

12. Phase-by-Phase Roadmap (implementation steps — follow sequentially)

Phase A — Project Setup & Core CLI

- Create Maven project skeleton, folder structure.
- Add dependencies: Gson, dotenv, JColor, mongodb-driver-sync, Apache Commons Text.
- Implement `Config` to read `.env` (API_KEY, MONGO_URI, default model).
- Implement `CLI` REPL skeleton (read, print).
- Implement `Formatter` for colored output and code block printing.

Deliverable: REPL that echoes input and supports `:help` / `:exit` .

Phase B — ChatClient & Basic Chat

- Implement `ChatClient` using `HttpClient` (sync POST).
- Build request/response mapping with Gson.
- Add `ChatService` to store messages in memory.
- On `:exit` , dump session to MongoDB via `MongoStorage` .

Deliverable: Chat with remote API and save session to DB.

Phase C — MongoDB Storage

- Implement `MongoStorage` (connect, insert session, list sessions, load session).
- Add `:history` and `:load <id>` commands.
- Ensure messages persisted with timestamps.

Deliverable: Persistent sessions saved & reloadable.

Phase D — Command Execution & FileManager

- Implement `CommandHandler` and `CommandRunner` with `ProcessBuilder` .
- Add `!` prefix handling. Securely capture output and show to user.
- Implement `FileManager` for `open` , `save` , `edit` flows (basic: read file, show contents; editing via AI return).

Deliverable: Run local commands and basic file operations.

Phase E — UX & Features

- Add JColor styling, code formatting, pagination for long outputs.
- Add `SuggestionEngine` for auto-suggestions/corrections.
- Implement streaming responses (optional advanced): show characters as they arrive.
- Add configuration commands: change model, set temperature, etc.
- Add command aliases & shortcuts.

Deliverable: Polished UX with helpful features.

Phase F — Hardening & Extras

- Add backups for file overwrites.
- Implement retries, rate-limit handling for API calls.
- Add search across sessions (text search via Mongo text index).
- Add optional encryption for stored API key.
- Package as runnable JAR and provide install instructions.

Deliverable: Productionizable CLI jar + README + tests.

13. Example `MongoStorage.java` (concept)

```
public class MongoStorage {
    private final MongoClient client;
    private final MongoCollection<Document> sessions;

    public MongoStorage(String uri) {
        client = MongoClient.create(uri);
        MongoDBDatabase db = client.getDatabase("chatgpt_cli");
        sessions = db.getCollection("sessions");
    }

    public String saveSession(Session session) {
```

```

        Document doc = new Document("_id", session.getId())
            .append("createdAt", session.getCreatedAt())
            .append("messages", session.getMessagesAsDocs());
        sessions.insertOne(doc);
        return session.getId();
    }

    public List<Session> listSessions(int limit) {
        // return list of sessions
    }

    public Session loadSession(String id) {
        // load document and convert to Session
    }
}

```

(Implementation detail: map between Java objects and `Document`)

14. Sample CLI Interaction Scripts

- `:history` → shows list of sessions with IDs and timestamps.
 - `:load session_2025...` → load and display conversation.
 - `!git status` → runs shell command and prints result.
 - `open src/Main.java` → prints file; then you can ask AI to "refactor this method".
 - `save changes.java` → saves assistant output as file.
-

15. Future Enhancements (nice to have)

- Streaming audio I/O (speak responses).
 - Plugin system to integrate external tools (lint, compile, run).
 - Integrate code execution sandboxes for multiple languages.
 - Optional remote server + web UI (Spring version) later.
-

16. Next Actions (what you can start coding immediately)

1. Initialize Maven project with dependencies.
 2. Create `.env.example` and implement `Config` class to read values.
 3. Implement `CLI` skeleton and `Formatter`.
 4. Implement `ChatClient` with a test using a mocked response (or one real call if you have API key).
 5. Stand up local MongoDB and implement `MongoStorage.saveSession()` and `listSessions()`.
-

If you want, I can now generate the **exact starter code** for Phase A + B:

- `pom.xml` (Maven) with dependencies
 - `Config.java` (dotenv load)
 - `Main.java` + `CLI.java` skeleton
 - `ChatClient.java` (basic HttpClient post + Gson mapping)
 - `MongoStorage.java` skeleton
-

14. Sample CLI Interaction Scripts

Below are example user–assistant interactions that demonstrate how the system should behave:

- `:history` → Displays list of past sessions with IDs and timestamps.
 - `:load session_2025_10_29` → Loads a saved chat session and prints it in formatted form.
 - `!git status` → Executes a shell command and prints terminal output.
 - `open src/Main.java` → Opens and displays a file in the console; the user can then ask AI to refactor or explain code.
 - `save changes.java` → Saves the assistant's latest response to a specified file.
-

15. Future Scope and Enhancements

Short-Term Enhancements

- **Improved command auto-suggestions** based on user history and frequent commands.
- **Error recovery and retry system** for failed OpenAI or database requests.
- **Customizable prompt templates** for different modes like “Code Review”, “Shell Assistant”, “Bug Fixing”, etc.

Medium-Term Enhancements

- **Streaming audio input/output** — the assistant can listen and speak replies.
- **File diff and merge support** — automatically compare edited files with originals before saving.
- **Multi-session windowing** — manage multiple chats in parallel within the same terminal.

Long-Term Enhancements

- **Plugin System:** Allow integration of external tools like linters, compilers, or testing frameworks.
- **Code Execution Sandboxes:** Run code snippets securely (supporting Java, Python, Bash, etc.) using Docker or process isolation.
- **AI Agent Mode:** Allow the assistant to execute a chain of commands (read → analyze → act → summarize).
- **Remote Server + Web UI:** Optionally extend the CLI to a Spring Boot or web-based version for remote usage.
- **Versioned Session Storage:** Enable restoring or branching chat sessions (like Git commits).

Implementation Detail (Mapping Between Java Objects and MongoDB Documents)

Each chat session and message will be serialized as a **Document** in MongoDB.

- Use the `org.bson.Document` or POJO mapping with `MongoCollection<ChatSession>`.
 - Each session includes metadata (sessionId, timestamp, context mode).
 - Each message contains sender, role (user/assistant/system), content, and timestamps. This schema allows structured queries, history search, and analytics later.
-

16. Next Steps (Immediate Implementation Plan)

Start with **Phase A + Phase B (Setup and Core CLI)**:

1. **Initialize a Maven project** with all dependencies (`gson` , `dotenv` , `mongo-java-driver` , `jansi` for colored output, etc.).
2. **Create `.env.example`** for storing the OpenAI API key and MongoDB URI securely.
3. **Implement `Config.java`** → loads environment variables safely.
4. **Implement `CLI.java`** → handles input, detects command prefixes (`:` , `!` , `open` , `save`), and prints output.
5. **Implement `ChatClient.java`** → uses `HttpClient` to make API requests and parse JSON responses with `Gson` .
6. **Implement `MongoStorage.java`** → handles `saveSession()` , `listSessions()` , and `loadSession()` methods.
7. **Add basic formatting utilities** (colored console output, indentation, etc.).

Once this skeleton is running, you'll already have a working **offline + online ChatGPT CL**